

OSSP-SKILL-WEEK1

Session 1

Task1: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

Source code:

```
GNU nano 7.2      Makefile *
CC = gcc
CFLAGS = -Wall -Wextra

all: main

main: main.c
    $(CC) $(CFLAGS) main.c -o main

clean:
    rm -f main
```

```
#include <stdio.h>

int main() {
    printf("Hello, OSSP Skill Week 1!\n");
    printf("GCC compilation is working successfully.\n");

    return 0;
}
```

Outputs:

```
make: Nothing to be done for 'all'.
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task1# ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task1# |
```

```
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task1# cd skill/Week1/Task1
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task1# nano main.c
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task1# gcc main.c -o main
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task1# ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task1# |
```

Verifying linux environment by uname-a:

```
root@LAPTOP-1C0JKKG6:~# uname -a
Linux LAPTOP-1C0JKKG6 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64 x86_64
4 x86_64 GNU/Linux
root@LAPTOP-1C0JKKG6:~# |
```

Checking GCC C compiler:

```
root@LAPTOP-1C0JKKG6:~# gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@LAPTOP-1C0JKKG6:~# |
```

Git version:

```
root@LAPTOP-1C0JKKG6:~# git --version
git version 2.43.0
root@LAPTOP-1C0JKKG6:~# |
```

Observation: I learned how to work with the Linux environment and verify system tools such as GCC and Git. I learned how to compile and execute a C program using GCC and understood the purpose of a Makefile for automating the compilation process. I also learned how Linux development projects can be organized and built using standard compiler and build commands.

Task2: To Analyse process abstraction, execute fork(), understand exec() family, analyse parent-child relationships, inspect process tree, practice system call tracing.

Source code:

```
Ubuntu
GNU nano 7.2 skill_week1_task2.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing ls using exec()\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }
    else {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Output:

```
root@LAPTOP-1C0JKKG6:~# gcc skill_week1_task2.c -o skill_week1_task2
root@LAPTOP-1C0JKKG6:~# ./skill_week1_task2
Parent Process Started
Parent PID: 638

Parent Process
Parent PID: 638
Child PID: 639
Parent waiting for child...

Child Process
Child PID: 639
Parent PID: 638
Executing ls using exec()
total 120
-rw-r--r-- 1 root root 53 Aug 8 06:04 destination.txt
drwxr-xr-x 5 root root 4096 Aug 11 06:52 ossp_2520090049
-rw-r--r-- 1 root root 41 Aug 8 08:32 sample.txt
-rwxr-xr-x 1 root root 16312 Aug 11 07:10 skill_week1_task2
-rw-r--r-- 1 root root 879 Aug 11 07:09 skill_week1_task2.c
-rw-r--r-- 1 root root 53 Aug 8 06:03 source.txt
-rwxr-xr-x 1 root root 16528 Aug 8 04:41 week1pt1
-rw-r--r-- 1 root root 1350 Aug 8 04:41 week1pt1.c
-rwxr-xr-x 1 root root 16224 Aug 8 06:03 week2pt1
-rw-r--r-- 1 root root 1208 Aug 8 06:02 week2pt1.c
-rwxr-xr-x 1 root root 16304 Aug 8 03:26 week3pt1
-rw-r--r-- 1 root root 1023 Aug 8 03:26 week3pt1.c
-rwxr-xr-x 1 root root 16136 Aug 8 03:33 week3pt2
-rw-r--r-- 1 root root 376 Aug 8 03:32 week3pt2.c
Child process completed.
root@LAPTOP-1C0JKKG6:~#
```

Observation: I learned how Linux creates and manages processes using fork(). I understood that exec() can replace the child process with another program, while wait() allows the parent process to wait until the child process finishes.

